



Photo by Michael Wieck on pixabay

Lecture 6

—

Decision Trees

Zitronenfalter (*Gonepteryx rhamni*)

- ♦ Der Zitronenfalter ist ein Schmetterling (Tagfalter) aus der Familie der Weißlinge (Pieridae). Die Falter erreichen eine Flügelspannweite von 50 bis 55 Millimetern.
- ♦ Der erste Schmetterling, der im Frühjahr durch die Gegend flattert, ist der Zitronenfalter. Er hat die erstaunliche Fähigkeit, im kalten Winter einen großen Teil seiner Körperflüssigkeit auszuscheiden und so Temperaturen von bis zu -20 °C überstehen zu können.
- ♦ Im Hochsommer fallen die Tiere dann in eine zweite Ruhephase, die Sommerruhe. So können sie als einziger deutscher Schmetterling bis zu zehn Monate alt werden.
- ♦ Die Raupen sind mattgrün gefärbt, wobei an den Seiten die Färbung schwächer ausgeprägt ist. Über den Beinchen verläuft auf jeder Seite ein heller tiefliegender mattweißer Längsstreifen. Diese Zeichnung, die dem Prinzip der Gegenschattierung folgt, dient der Raupe als Tarnung vor Feinden, wie zum Beispiel Singvögeln.
- ♦ Männliche Exemplare sind dabei zitronengelb, während weibliche Tiere eher grünlichgelb gefärbt sind.

Quellen:

* <https://de.wikipedia.org/wiki/Zitronenfalter>

* <https://www.plantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten>



Classification – Decision Trees & Ensemble Learning

1. Decision Trees
2. Bagging
3. Random Forests and Extra Trees
4. Ensemble Learning



History of Decision Trees

- ◆ Earliest Ideas
 - AID (Morgan and Sonquist, 1963), CHAID (Kass, 1964)
- ◆ Mainstream
 - CART (Breiman et al., 1984), ID3 (Quinlan, 1986), C4.5 (Quinlan, 1993)
- ◆ Bagging and Boosting
 - Random forest (Breiman, 2001), XGBoost (Chen, 2014), LightGBM (Microsoft, 2017), CatBoost (Yandex, 2017), HistGradientBoosting (sklearn, 2019)



Decision Trees - Usage

- Example: Decision Tree for the Iris dataset: classify different Iris flowers (Setosa, Versicolour, and Virginica = target) based on the predictors: Sepal Length, Sepal Width, Petal Length and Petal Width.



Iris setosa

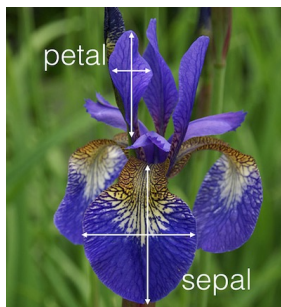


Iris versicolor

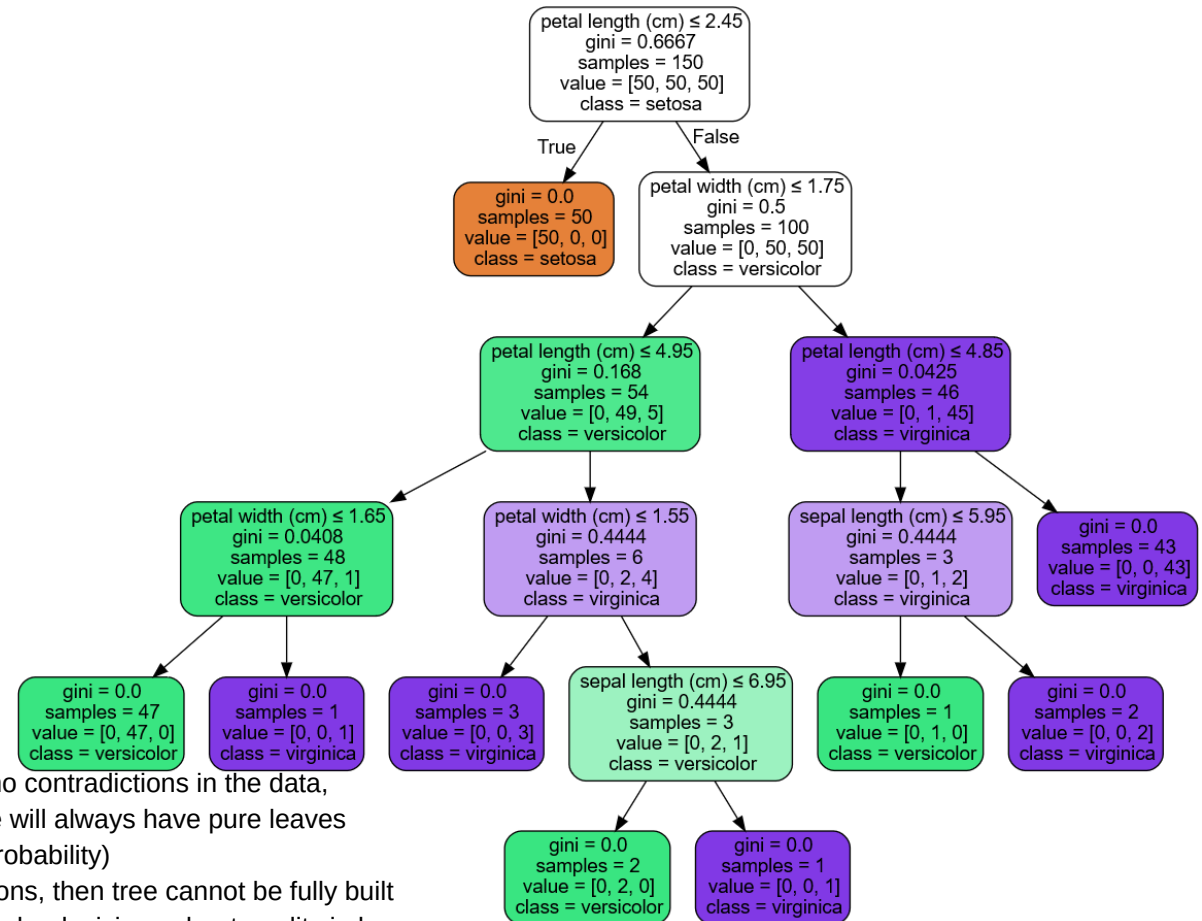


Iris virginica

source: <http://www.stats.uwo.ca/faculty/aim/2017/sdm/notebooks/16-IrisPCA/IrisPCA.html>



source: http://sebastianraschka.com/Articles/2014_python_lda.html



if there are no contradictions in the data,
then the tree will always have pure leaves
(i.e. 100% probability)
if contradictions, then tree cannot be fully built
or need to make decisions about quality in leaves
based on e.g. Gini Impurity

source: <http://scikit-learn.org/stable/modules/tree.html>

fully constructing a tree runs into the danger of overfitting
> pruning/leaf eval necessary

during tree application, data goes as far through the tree as possible even if it has features missing,
will return probability for classification instead of definite results



Top-Down construction of a Decision Tree

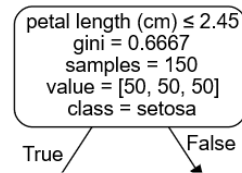
- ◆ **Recursive** Algorithm
 - Top-down, i.e. starting at the root
- ◆ Node chooses the split that gives
 - the best **impurity reduction** (**classification**)
 - **error reduction** (**regression**)

subject to what kinds of splits are allowed for the feature type

- ◆ **Candidate splits** are generated **depending on feature type**



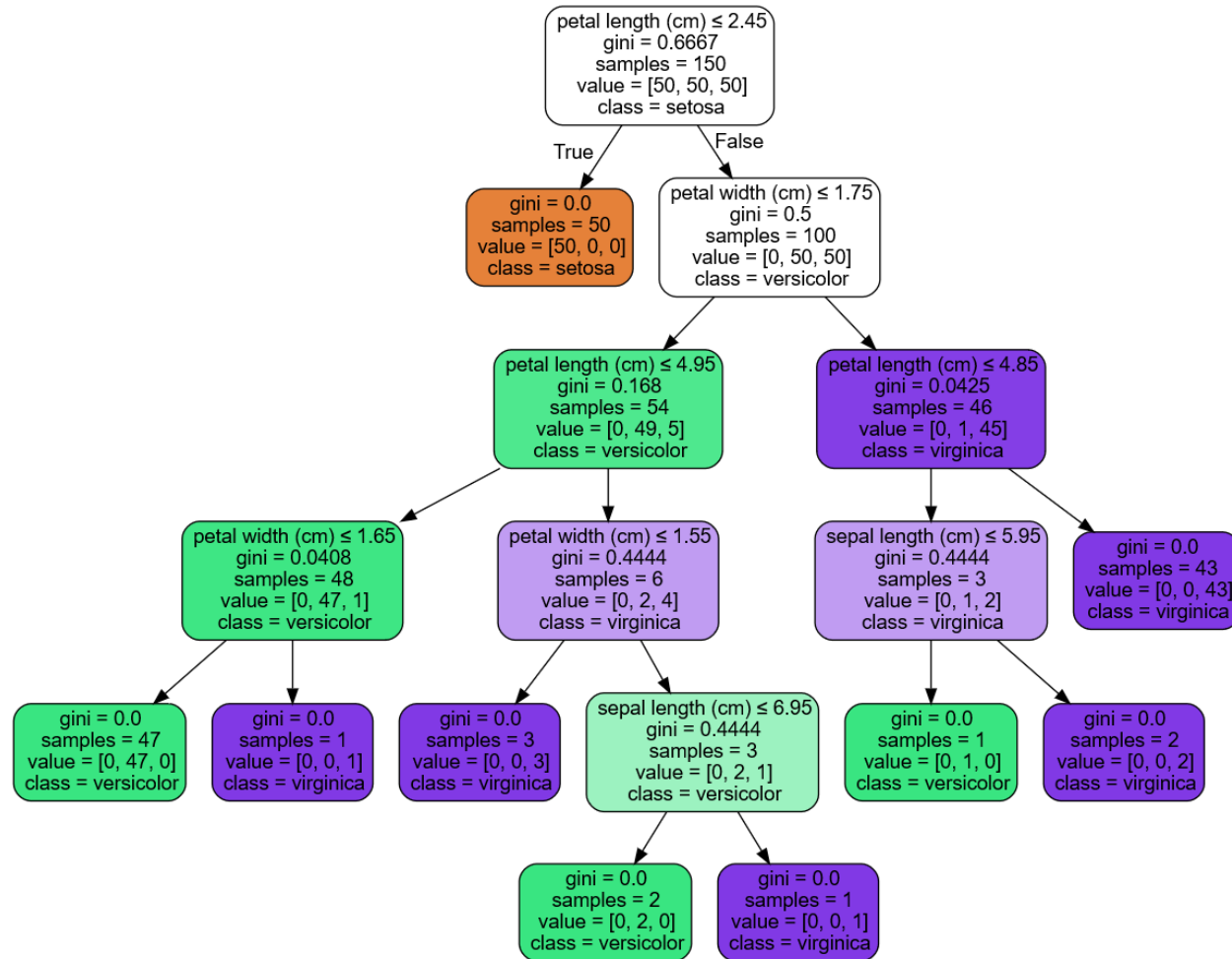
Decision Trees – Construction



source: <http://scikit-learn.org/stable/modules/tree.html>



Decision Trees – Construction



source: <http://scikit-learn.org/stable/modules/tree.html>



Split Criteria in Decision Trees: How Do Trees Decide Where to Split?

Key Split Criteria (impurity measures) for Classification

◆ Gini Impurity

- Measures how often a randomly chosen element would be misclassified if it were randomly labeled according to the distribution in the node
- Formula: $Gini = 1 - \sum p_i^2$
- Lower Gini $\rightarrow$ More pure nodes
- Default for Classification in sklearn

◆ Information Gain (Entropy-Based Split)

- Based on Shannon entropy, which measures uncertainty in a dataset
- Formula for entropy: $H(X) = -\sum p_i \log_2(p_i)$
- Information Gain = Reduction in entropy after a split: $IG = H(Parent) - \sum \left(\frac{N_{child}}{N_{parent}} H(Child) \right)$
- Higher Information Gain $\rightarrow$ More useful split



Generation of the Split Point Candidates (1)

♦ Numerical (continuous) features

- Split on threshold: $x \leq t$ vs. $x > t$
- Candidate thresholds
 - Sort unique values
 - Consider thresholds between consecutive distinct values, typically midpoints
- Compute gain for each potential threshold, pick best
- Note: Some tree variants can do multiway numeric splits, but most are binary

♦ Ordinal features

- Two common treatments:
 - Treat as numerical with order (as numeric thresholds, but thresholds are between adjacent levels)
 - Treat as categorical but restrict to order-respecting splits
- Typically, do not allow arbitrary subsets, because that breaks the meaning of ordinal order

♦ Binary features

- Split $x = 0$ vs. $x = 1$ (trivial)
- Note: if it's numeric 0/1, many implementations just treat it as numeric; same result!



Generation of the Split Point Candidates (2)

♦ **Categorical features:** Multiple Options

1) **Multiway split** (ID3 / C4.5)

- Split form: one child per category value
 - Pros: very expressive
 - Cons: can overfit, especially when m is large; information gain tends to favour high-cardinality features

2) **Binary subset split** (CART)

- Split form: choose a subset S of categories: $x \in S$ vs. $x \notin S$
- Non-empty proper subsets: $2^{m-1} - 1$ candidates: too many when cardinality m is large
→ use heuristics!

3) **One-hot encoded**

- Each category becomes a binary indicator
- Tree splits on those binaries.
- Works well
 - common in modern ML pipelines
 - but can change the search space; may behave differently than native categorical handling



Decision Trees - Summary



- ♦ Fast – can handle large datasets
- ♦ Can handle multiple targets and mixed predictors
- ♦ Redundant predictors ignored automatically
- ♦ Can handle missing data
- ♦ Provide probability of prediction (is usually 100% when reaching leaf)
- ♦ Small trees easy to interpret



- ♦ Large trees hard to interpret
- ♦ Overfitting hard to control (boosting good for counteracting?)
- ♦ Prediction accuracy often poor (when only one tree)

Curse of Dimensionality does not apply here
tree evals unnecessary features but will not choose them if they do not properly represent the data



Exercises

Exercise 1

Decision Trees





Classification – Decision Trees & Ensemble Learning

1. Decision Trees
2. Bagging
3. Random Forests and Extra Trees
4. Ensemble Learning



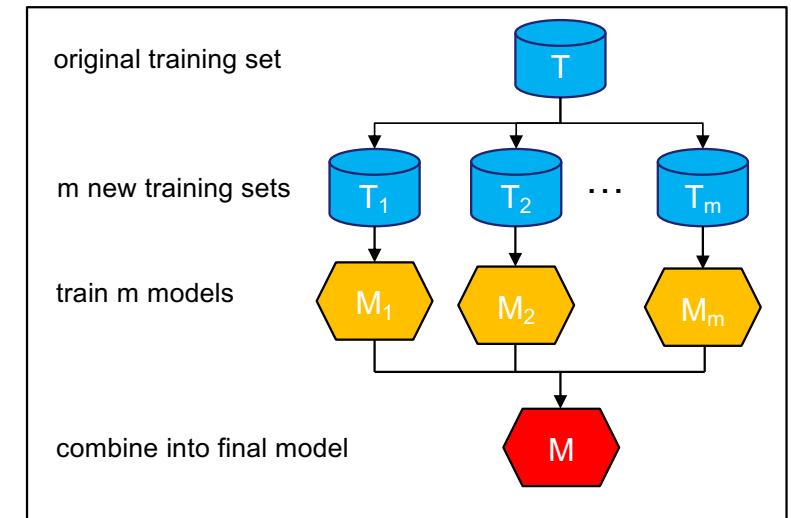
Bagging

♦ Bagging (Bootstrap Aggregation) [Breiman, 1994]

- “why train only on decision tree, when you can have many”
- Combine multiple high-variance Decision Trees (can be used for other models as well)
- Reduce variance/overfitting while keeping low-bias and improve stability of the model

♦ Algorithm

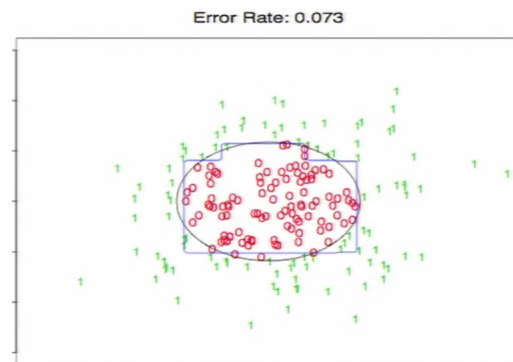
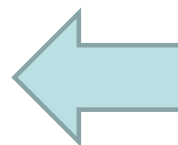
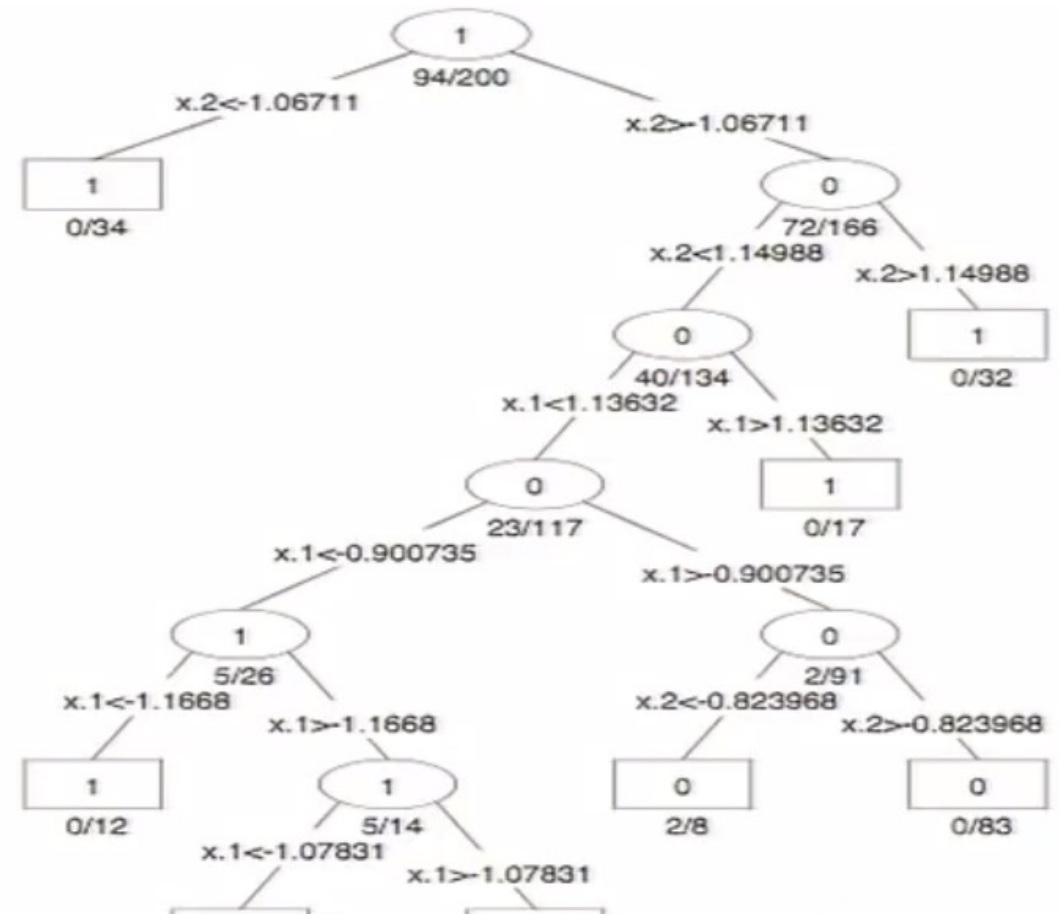
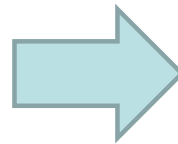
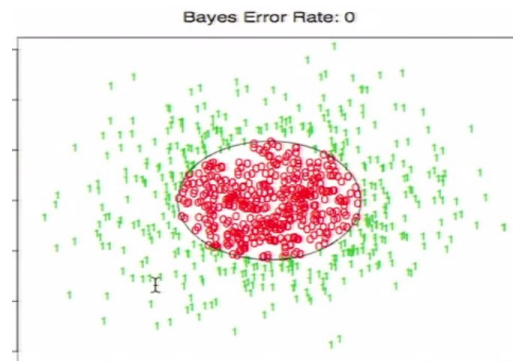
- Given: training set T of size n
- Generate m new training sets T_i of size n' by sampling with replacement (“Bootstrap Samples”)
 - uniform random sampling with replacement $\rightarrow$ some observations may be repeated in each T_i
 - $n' = n \rightarrow T_i$ expected to contain $\left(1 - \frac{1}{e}\right) \approx 63.2\%$ of the unique observations of T (for large n)
- Train m models using the m bootstrap samples T_i
- Combine the results
 - Usually: Majority vote (for classification) / Averaging (for regression)





Bagging – Example Data Set (2-dimensional, no noise)

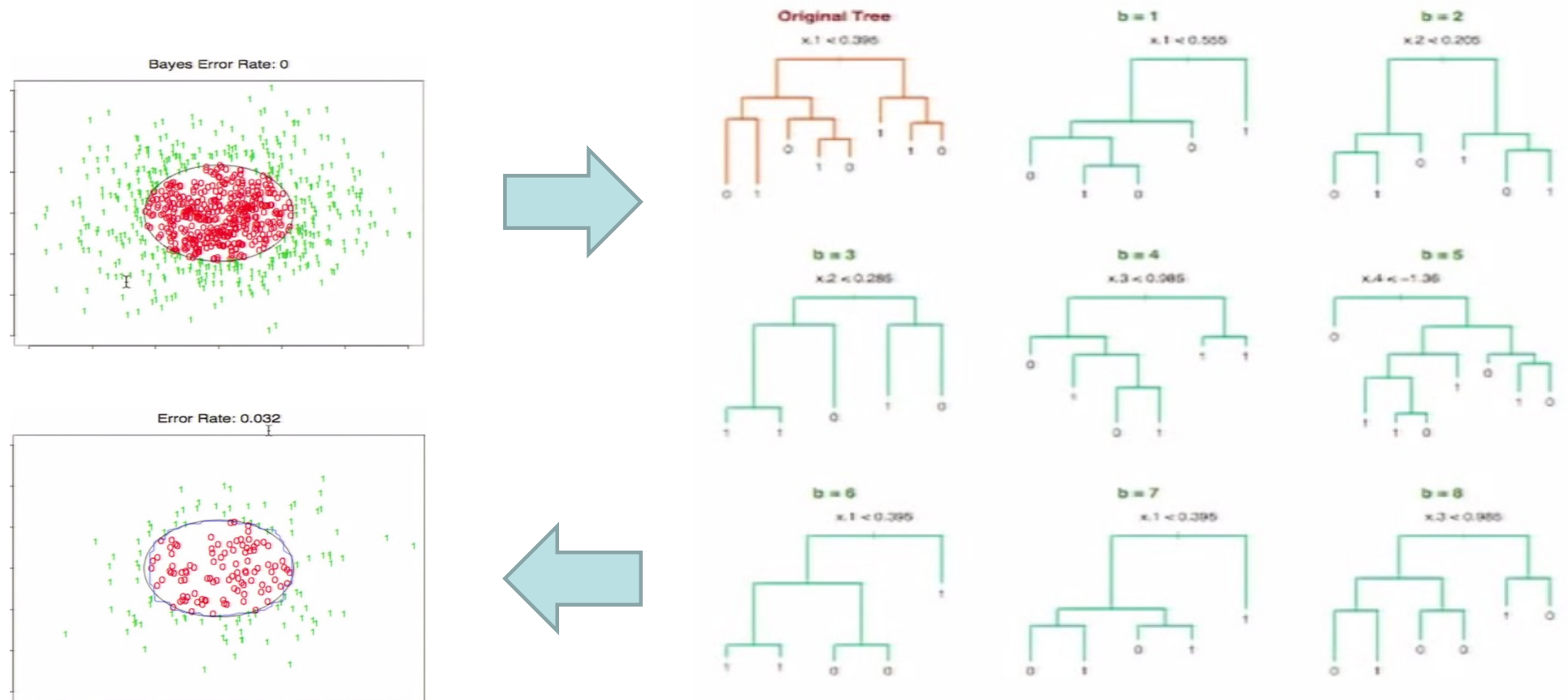
source: <https://www.youtube.com/watch?v=wPqtzj5VZus>





Bagging – Example Data Set (2-dimensional, no noise)

source: <https://www.youtube.com/watch?v=wPqtzj5VZus>



the more trees, the more it goes into overfitting
choosing the number of trees is a hyperparam

problem: there are certain significant features that will strongly influence the tree, resulting in trees that all look the same for the root nodes and only differ further down where there are less samples with muddier results



Classification – Decision Trees & Ensemble Learning

1. Decision Trees
2. Bagging
3. Random Forests and Extra Trees
4. Ensemble Learning



Why Move Beyond Bagging?

♦ Why Random Forests and Extra Trees?

- Decision trees are **prone to overfitting**
- Bagging reduces variance but still has **correlated trees**
- We need **even more randomness** for better generalization

♦ What is a Random Forest?

- Ensemble of multiple decision trees
- Uses bootstrapped datasets (sampling with replacement)
- At each node, only a **random subset of features is considered for splitting**

➔ Smarter Bagging: Reduces overfitting and improves generalization.



How Random Forests Work

♦ Building a Random Forest

- Draw **bootstrap samples** from the dataset
- Train a decision tree on each sample:
At each split decision, select a **random subset of the features** for consideration (“feature bagging”)
(rule of thumb: for p predictors, select $\lfloor \sqrt{p} \rfloor$ for classification / select $\lfloor p/3 \rfloor$ for regression) values are floored
choosing how many features to eval is once again hyperparam but above is used often as rule of thumb
- Aggregate predictions via **majority vote** (classification) or **averaging** (regression)

- ### ♦ Motivation: Correlation of the trees in a simple bootstrap sample: if one (or a few) predictors are very strong indicators for the target variable, these predictors will be selected in many of the trees, causing the trees to become correlated and Bagging to overfit

random forest is very popular, gives better result, cheap and can be parallelized perfectly



Important Hyperparameters of Random Forests

- ◆ Random Forests are powerful, but tuning their hyperparameters is crucial!
- ◆ Tuning a Random Forest
 - `n_estimators`: Number of trees in the forest
 - `max_features`: Number of features to consider at each split
 - `max_depth`: Maximum depth of each tree
 - `min_samples_split`: Minimum samples required to split a node



Extra Trees

♦ Extra Trees: Even More Randomness

Similar to Random Forests, but with two key differences:

- No bootstrapping – full dataset is used for each tree
- Random split points – instead of choosing the best split, a random split is selected

♦ Impact on Bias

- Random Forests: optimize the split at each node → capture more patterns in the data
→ bias slightly lower
- Extra Trees: arbitrary split points (more random) → suboptimal splits → do not fit the data as well
→ bias slightly higher

♦ Impact on Variance

- Random Forests: reduce variance significantly compared to a single decision tree
- Extra Trees: even lower variance because their increased randomness prevents overfitting



Comparison

♦ Comparison: When to Use What?

Feature	Random Forest	Extra Tree
Bootstrapping	Yes	No
Split Selection	Best Split	Random Split
Variance Reduction	High	Even Higher
Bias	Slightly Lower	Slightly Higher
Speed	Slower	Faster

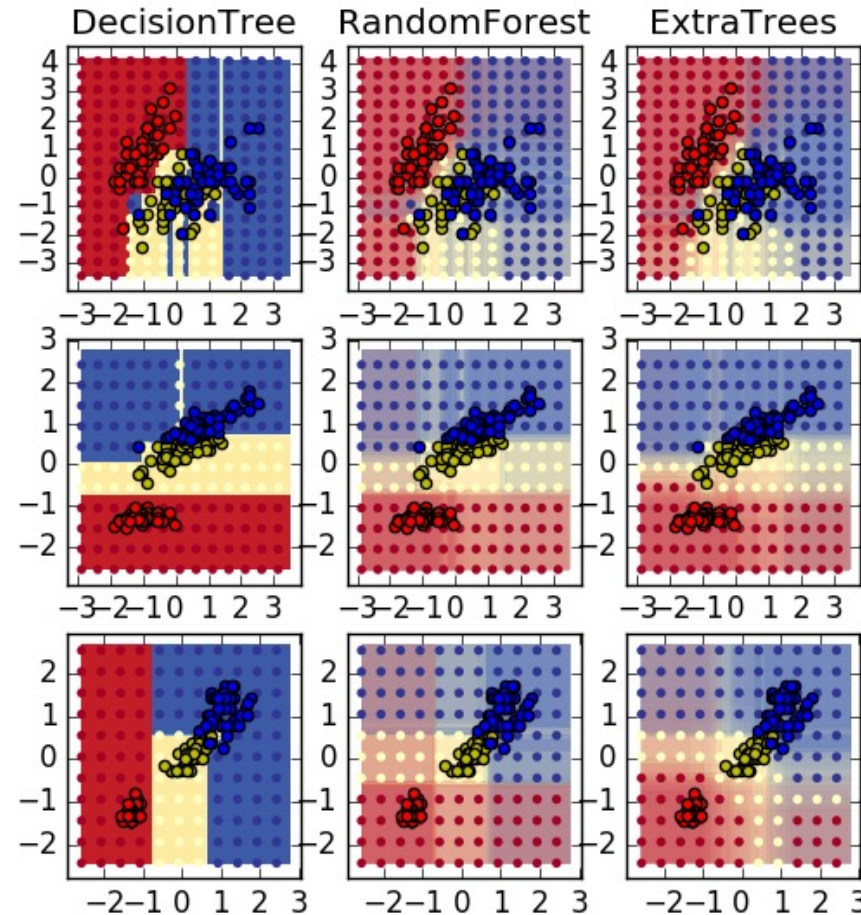
♦ Rules of Thumb

- If you prioritize accuracy and don't mind more computation, Random Forests are usually better
- If you need faster training and extra variance reduction, Extra Trees might be preferable
- If the dataset is very noisy, Extra Trees can prevent overfitting better than Random Forests



Random Forests and Extra Trees Example

- ♦ Example: Combinations of predictors for the Iris dataset:
 - Sepal Length vs. Sepal Width
 - Sepal Length vs. Petal Length
 - Petal Length vs. Petal Width.



source: http://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_iris.html



Let's do 4
short questions

Every question has
4 statements:
3 of which are correct,
1 of which is wrong.

Pick the wrong one!



Question 1 – Decision Trees

Which statement is WRONG

- A) Every observation in the training data is associated with exactly one leaf node
- B) Decision trees can handle contradicting training observations
- C) A fully trained decision tree will classify all training observations **correctly**
if there is contradicting data in the set, then it cannot classify all of it correctly
- D) Even fully trained decision trees cannot necessarily classify all new observations



Question 2 – Bagging is a technique that does...

Which statement is WRONG

- A) reduce variance
- B) reduce overfitting
- C) use high-bias Decision Trees
- D) improve stability



Question 3 – Random Forests do

Which statement is WRONG

- A) use a bootstrap sample for the training data sets
- B) use feature bagging when selecting predictors
- C) increase the randomness of the resulting trees
- D) draw split points **at random** for each candidate predictor at each split decision this is extra tree



Question 4 – Bias-Variance Trade-off

Which statement is WRONG

- A) High bias models tend to underfit
- B) High bias models tend to be sensitive to small fluctuations in the training set
- C) Low variance models tend to be simple
- D) Linear regression has strong assumptions about the data



Classification – Decision Trees & Ensemble Learning

1. Decision Trees
2. Bagging
3. Random Forests and Extra Trees
4. Ensemble Learning



What is Ensemble Learning?

Ensemble Learning

- ◆ Combines multiple models to make better predictions
- ◆ Aims to improve accuracy, robustness, and generalization
- ◆ Works by reducing bias, variance, or both
- ◆ Inspired by the Wisdom of the Crowd principle
- ➔ Meta-algorithm: Ensemble Learning is itself a learning algorithm producing a prediction



Why Does Ensemble Learning Work?

The Wisdom of the Crowd

- ◆ Example: Estimating the weight of an ox (Galton's experiment, 1907).
- ◆ Average of many individual estimates: more accurate than most individual guesses
- ◆ Same principle applied in machine learning: combine multiple weak models into a strong model



Source: <https://pixabay.com/de/photos/highlander-stier-ochse-das-vieh-1128786/>



Types of Ensemble Learning

Types of Ensemble Learning

◆ Averaging-based methods

- Reduce variance
- Examples: Bagging, Random Forests, Extra Trees

◆ Boosting-based methods

- Reduce bias
- Example: AdaBoost, Gradient Boosting, XGBoost

◆ Stacking & Blending

- Combine different models using a meta-learner
- Stacking: multiple base models and a meta-learner to combine their predictions
 - Frequently: logistic regression as combiner
- Blending: simpler version of stacking: predictions from base models are weighted and averaged



Why Use Ensemble Learning?

♦ Benefits of using Ensemble Learning

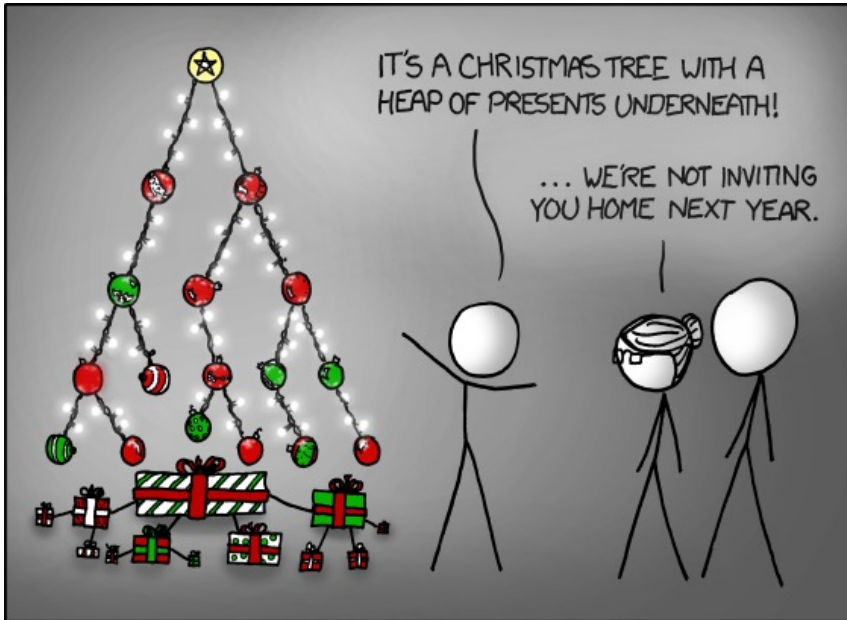
- **Better Accuracy:** Combining models improves predictive performance
- **Less Overfitting:** Bagging reduces variance, Boosting reduces bias
- **More Robustness:** Handles different types of errors better (Tune the Bias-Variance-Tradeoff)
- **Versatility:** Works for both classification and regression

➔ Very powerful technique!

♦ Key Parameter: Ensemble Size

- Generally hard to determine
- Use train/test or statistical test (→ hyperparameter to be tuned)

♦ Choosing the right ensemble method depends on dataset size, noise, and performance needs



Source: <https://xkcd.com/835/>

Further Reading:

Ensemble methods in scikit-learn

- <http://scikit-learn.org/stable/modules/ensemble.html>

Decision Trees

- ID3: Quinlan, J. R. 1986. Induction of Decision Trees. Mach. Learn. 1, 1 (Mar. 1986), 81–106
- C4.5: Quinlan, J. R. C4.5: Programs for Machine Learning. Morgan Kaufmann Publishers, 1993.

Random Forests

- Breiman, Leo (2001). "Random Forests". Machine Learning. **45** (1): 5–32. doi:10.1023/A:1010933404324.
- Ho, Tin Kam (1995). Random Decision Forests. Proceedings of the 3rd International Conference on Document Analysis and Recognition, Montreal, QC, 14–16 August 1995. pp. 278–282.



Key Takeaways

- ◆ Top-Down Induction of Decision Trees
- ◆ Split Criteria
 - Gini Impurity
 - Entropy-based Information Gain
- ◆ Bagging, Random Forests, ExtraTrees – combine multiple high-variance/low-bias models
- ◆ Ensemble Learning

